

Lista 5 - Sztuczna inteligencja

Mikołaj Kamiński

Ładowanie zbioru danych

```
from google.colab import userdata, files, drive
from huggingface_hub import login
from datasets import load_dataset
from transformers import pipeline, AutoModelForCausalLM, AutoTokenizer
from langchain_huggingface import HuggingFacePipeline
from langchain_core.prompts import PromptTemplate
import torch
import numpy as np
import pandas as pd
import json
import logging
import shutil
import gc
from collections import Counter
from sklearn.metrics import confusion_matrix, accuracy_score, f1_score, classification_report, recall_score, precision_score
from datetime import datetime
from langchain_core.output_parsers import JsonOutputParser
from pydantic import BaseModel, Field
from pathlib import Path
import os
```

```
hf_token = userdata.get('HF_TOKEN')
login(token=hf_token)
drive.mount("/content/drive")
os.makedirs("/content/drive/MyDrive/Colab_results/Lista5SI/hf_cache", exist_ok=True)
os.environ["HF_HOME"] = "/content/drive/MyDrive/Colab_results/Lista5SI/hf_cache"
os.environ["HF_HUB_CACHE"] = "/content/drive/MyDrive/Colab_results/Lista5SI/hf_cache/hub"
os.environ["TRANSFORMERS_CACHE"] = "/content/drive/MyDrive/Colab_results/Lista5SI/hf_cache/transformers"
```

Mounted at /content/drive

```
dataset = load_dataset("allegro/klej-polemo2-in", split="test")
print(f"Text:{dataset[0]['sentence']}")
print(f" Label :{ dataset [0]['target']}")
```

```
README.md: 100% 6.01k/6.01k [00:00<00:00, 160kB/s]
train.csv: 100% 4.88M/4.88M [00:00<00:00, 21.9MB/s]
valid.csv: 100% 602k/602k [00:00<00:00, 11.5MB/s]
test.csv: 100% 591k/591k [00:00<00:00, 6.05MB/s]

Generating train split: 100% 5783/5783 [00:00<00:00, 11440.91 examples/s]
Generating validation split: 100% 723/723 [00:00<00:00, 10771.20 examples/s]
Generating test split: 100% 722/722 [00:00<00:00, 8007.30 examples/s]
Text:Leczyła m się u niej parę lat i nic mi nie pomogła , a jak zmieniała m lekarza po krótkim czasie zobaczyła m już poprawę
Label :__label__meta_minus_m
```

```
filtered_dataset = dataset.filter(lambda example: example['target'] != '__label__meta_amb')
print(f"Liczba przed filtrowaniem: {dataset.num_rows}")
print(f"Liczba po usunięciu 'ambiguous': {filtered_dataset.num_rows}\n")

class_counts = Counter(filtered_dataset['target'])

print("Zrównoważenie klas:")
for label, count in class_counts.items():
    percentage = (count / filtered_dataset.num_rows) * 100
    print(f"{label}: {count} ({percentage:.2f}%)")
print("\n")

def compute_lengths(example):
    return {
        "char_length": len(example["sentence"]),
        "word_count": len(example["sentence"].split())
    }
```

```

dataset_with_lengths = filtered_dataset.map(compute_lengths)

word_counts = dataset_with_lengths["word_count"]
char_counts = dataset_with_lengths["char_length"]

print("Statystyki długości tekstów (LICZBA SŁÓW):")
print(f"Średnia: {np.mean(word_counts):.2f}")
print(f"Min: {np.min(word_counts)}, Max: {np.max(word_counts)}")
print(f"Mediana: {np.median(word_counts)}")
print("\n")

print("Statystyki długości tekstów (LICZBA ZNAKÓW):")
print(f"Średnia: {np.mean(char_counts):.2f}")
print(f"Min: {np.min(char_counts)}, Max: {np.max(char_counts)}")
print(f"Mediana: {np.median(char_counts)}")

```

Filter: 100% 722/722 [00:00<00:00, 9072.11 examples/s]

Liczba przed filtrowaniem: 722
Liczba po usunięciu 'ambiguous': 614

Zrównoważenie klas:

__label__meta_minus_m: 300 (48.86%)
__label__meta_zero: 117 (19.06%)
__label__meta_plus_m: 197 (32.08%)

Map: 100% 614/614 [00:00<00:00, 1769.99 examples/s]

Statystyki długości tekstów (LICZBA SŁÓW):
Średnia: 132.63
Min: 6, Max: 440
Mediana: 120.0

Statystyki długości tekstów (LICZBA ZNAKÓW):
Średnia: 759.39
Min: 29, Max: 2469
Mediana: 694.0

▼ Testowanie modeli encoder only

```

TIMESTAMP = datetime.now().strftime("%Y%m%d_%H%M%S")
RESULTS_DIR = Path("/content/drive/MyDrive/Colab_results/Lista5SI/results_encoder") / TIMESTAMP
MODELS = [
    "Voicelab/herbert-base-cased-sentiment",
    "bardsai/twitter-sentiment-pl-base",
    "tabularisai/multilingual-sentiment-analysis",
]

BATCH_SIZE = 16
PARAM_GRID = [
    {"batch_size": BATCH_SIZE, "truncation": True, "max_length": 64},
    {"batch_size": BATCH_SIZE, "truncation": True, "max_length": 128},
    {"batch_size": BATCH_SIZE, "truncation": True, "max_length": 256},
    {"batch_size": BATCH_SIZE, "truncation": True, "max_length": 512},
]

KNOWN_ID2LABEL = {
    "Voicelab/herbert-base-cased-sentiment": {
        0: "negative",
        1: "neutral",
        2: "positive",
    },
    "bardsai/twitter-sentiment-pl-base": {
        0: "positive",
        1: "neutral",
        2: "negative",
    },
    "tabularisai/multilingual-sentiment-analysis": {
        0: "Very Negative",
        1: "Negative",
        2: "Neutral",
        3: "Positive",
        4: "Very Positive",
    },
},
}
_UNKNOWN_LABELS: set[str] = set()

```

```

def setup_results_dirs() -> None:
    for subdir in ("predictions", "metrics", "logs", "reports"):

```

```

        (RESULTS_DIR / subdir).mkdir(parents=True, exist_ok=True)

def setup_logging() -> Path:
    log_path = RESULTS_DIR / "logs" / f"ewaluacja_{TIMESTAMP}.log"
    logging.basicConfig(
        filename=str(log_path),
        level=logging.INFO,
        format="%(asctime)s [%(levelname)s] %(message)s",
        encoding="utf-8",
    )
    return log_path

def safe_name(model_name: str) -> str:
    return model_name.replace("/", "_")

def normalize_id2label(raw_id2label: dict) -> dict[int, str]:
    return {int(k): v for k, v in raw_id2label.items()}

def _to_polemo_label(label_name: str) -> str | None:
    """Mapuje nazwę etykiety modelu na klasę PolEmo2."""
    label = label_name.lower().strip()

    if label == "very negative" or label == "negative":
        return "__label__meta_minus_m"
    if label == "very positive" or label == "positive":
        return "__label__meta_plus_m"
    if label == "neutral":
        return "__label__meta_zero"
    if "negatywn" in label or "minus" in label:
        return "__label__meta_minus_m"
    if "pozytywn" in label or "plus" in label:
        return "__label__meta_plus_m"

    return None

def map_label(
    model_label: str,
    id2label: dict[int, str] | None = None,
    model_name: str | None = None,
) -> str:
    """Przetwarza etykiety z różnych modeli na standard PolEmo2.0."""
    original = str(model_label).strip()
    label_key = original.upper()

    if label_key.startswith("LABEL_"):
        idx = int(label_key.split("_")[1])
        lookup = id2label or (KNOWN_ID2LABEL.get(model_name) if model_name else None)
        if lookup and idx in lookup:
            mapped = _to_polemo_label(lookup[idx])
            if mapped:
                return mapped

    mapped = _to_polemo_label(original)
    if mapped:
        return mapped

    if original not in _UNKNOWN_LABELS:
        _UNKNOWN_LABELS.add(original)
        logging.warning(
            "Nieznana etykieta modelu: %s -> przypisano neutral", original
        )
    return "__label__meta_zero"

```

```

def evaluate_model(
    dataset,
    model_name: str,
    param_id: str,
    batch_size: int = 16,
    truncation: bool = True,
    max_length: int = 128,
    save_csv: bool = True,
) -> dict:
    assert "__label__meta_amb" not in set(dataset["target"])
    device = 0 if torch.cuda.is_available() else -1
    logging.info(
        "Start ewaluacji: model=%s, param_id=%s, batch_size=%d, max_length=%d",
        model_name,

```

```

        param_id,
        batch_size,
        max_length,
    )

classifier = pipeline("text-classification", model=model_name, device=device)
id2label = normalize_id2label(classifier.model.config.id2label)

texts = list(dataset["sentence"])
true_labels = list(dataset["target"])

unique_labels = sorted(list(set(true_labels)))

raw_predictions = classifier(
    texts,
    batch_size=batch_size,
    truncation=truncation,
    max_length=max_length,
)
mapped_predictions = [
    map_label(pred["label"], id2label=id2label, model_name=model_name)
    for pred in raw_predictions
]

# Use the labels parameter to restrict metrics to the filtered classes
accuracy = accuracy_score(true_labels, mapped_predictions)
f1 = f1_score(true_labels, mapped_predictions, average="macro", labels=unique_labels)
recall = recall_score(true_labels, mapped_predictions, average="macro", labels=unique_labels)
precision = precision_score(true_labels, mapped_predictions, average="macro", labels=unique_labels)

run_timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
safe_model = safe_name(model_name)
pipeline_kwargs = {
    "batch_size": batch_size,
    "truncation": truncation,
    "max_length": max_length,
}

evaluation_summary = {
    "model": model_name,
    "param_id": param_id,
    "batch_size": batch_size,
    "truncation": truncation,
    "max_length": max_length,
    "kwargs": pipeline_kwargs,
    "id2label": {str(k): v for k, v in id2label.items()},
    "timestamp": run_timestamp,
    "ogolne_metryki": {
        "accuracy": accuracy,
        "f1_macro": f1,
        "recall_macro": recall,
        "precision_macro": precision,
    },
    "detailed_report": classification_report(
        true_labels, mapped_predictions, labels=unique_labels, output_dict=True
    ),
}

report_text = classification_report(true_labels, mapped_predictions, labels=unique_labels)
report_path = RESULTS_DIR / "reports" / f"{safe_model}_{param_id}_{run_timestamp}.txt"
with open(report_path, "w", encoding="utf-8") as f:
    f.write(f"Model: {model_name}\n")
    f.write(f"Parametry: param_id={param_id}, batch_size={batch_size}, ")
    f.write(f"truncation={truncation}, max_length={max_length}\n")
    f.write(f"Accuracy: {accuracy:.4f}\n")
    f.write(f"F1 macro: {f1:.4f}\n\n")
    f.write("Szczegółowy raport (Quality per class):\n")
    f.write(report_text)

json_path = RESULTS_DIR / "metrics" / f"{safe_model}_{param_id}_{run_timestamp}.json"
with open(json_path, "w", encoding="utf-8") as f:
    json.dump(evaluation_summary, f, indent=4, ensure_ascii=False)

if save_csv:
    df = dataset.to_pandas()
    df["raw_prediction"] = [pred["label"] for pred in raw_predictions]
    df["mapped_prediction"] = mapped_predictions
    df["prediction_score"] = [pred["score"] for pred in raw_predictions]
    csv_path = RESULTS_DIR / "predictions" / f"wyniki_{safe_model}_{param_id}.csv"
    df.to_csv(csv_path, index=False, encoding="utf-8")

logging.info(

```

```

        "Zakonczono: model=%s, param_id=%s, accuracy=%.4f, f1_macro=%.4f",
        model_name,
        param_id,
        accuracy,
        f1,
    )

    return {
        "model": model_name,
        "param_id": param_id,
        "batch_size": batch_size,
        "truncation": truncation,
        "max_length": max_length,
        "accuracy": accuracy,
        "f1_macro": f1,
        "recall_macro": recall,
        "precision_macro": precision,
        "timestamp": run_timestamp,
    }

```

```

def run_experiments(dataset, models: list[str], param_grid: list[dict]) -> pd.DataFrame:
    summary_rows = []
    total = len(models) * len(param_grid)
    run_num = 0

    for model_name in models:
        for params in param_grid:
            run_num += 1
            trunc_tag = "trunc" if params["truncation"] else "notrunc"
            param_id = f"ml{params['max_length']}_{trunc_tag}"
            logging.info("Eksperyment %d/%d", run_num, total)
            result = evaluate_model(
                dataset,
                model_name,
                param_id=param_id,
                batch_size=params["batch_size"],
                truncation=params["truncation"],
                max_length=params["max_length"],
            )
            summary_rows.append(result)

    summary_df = pd.DataFrame(summary_rows)
    summary_path = RESULTS_DIR / f"porownanie_ewaluacji_{TIMESTAMP}.csv"
    summary_df.to_csv(summary_path, index=False, encoding="utf-8")
    logging.info("Zapisano zbiorcze porownanie: %s", summary_path)
    return summary_df

def zip_and_download(results_dir: Path = RESULTS_DIR) -> str:
    zip_base = f"wyniki_ewaluacji_{TIMESTAMP}"
    zip_path = shutil.make_archive(zip_base, "zip", str(results_dir))
    files.download(zip_path)
    print(f"Zapisano i pobrano: {zip_path}")
    return zip_path

```

```

setup_results_dirs()
setup_logging()
run_experiments(filtered_dataset, MODELS, PARAM_GRID)
zip_and_download()

```

▼ Testowanie LLM

```

TIMESTAMP = datetime.now().strftime("%Y%m%d_%H%M%S")
RESULTS_DIR = Path("/content/drive/MyDrive/Colab_results/List5SI/results_llm") / TIMESTAMP

MODELS = [
    "microsoft/Phi-3-mini-4k-instruct",
    'Qwen/Qwen2.5-0.5B-Instruct',
    'Qwen/Qwen2.5-1.5B-Instruct'
]

GENERATION_BATCH_SIZE = 8
RUN_SANITY_CHECK = False
SANITY_CHECK_N = 5
DOWNLOAD_EVERY_N = 1

DECODING_GRID = [
    {"do_sample": False, "temperature": None, "max_new_tokens": 64},

```

```

{"do_sample": True, "temperature": 0.3, "max_new_tokens": 64},
{"do_sample": True, "temperature": 1.0, "max_new_tokens": 64},
{"do_sample": False, "temperature": None, "max_new_tokens": 20},
]

_UNKNOWN_SENTIMENTS: set[str] = set()
class SentimentOut(BaseModel):
    sentiment: str = Field(description="jedna z: positive, negative, neutral")

OUTPUT_PARSER = JsonOutputParser(pydantic_object=SentimentOut)
FORMAT_INSTRUCTIONS = OUTPUT_PARSER.get_format_instructions()

PROMPT_ZERO_SHOT = """Jesteś ekspertem od analizy sentymentu tekstów w języku polskim.
Określ sentyment poniższego tekstu jako positive, negative lub neutral.

{format_instructions}

Tekst do analizy:
{text}"""

PROMPT_FEW_SHOT = """Jesteś ekspertem od analizy sentymentu tekstów w języku polskim.
Określ sentyment poniższego tekstu jako positive, negative lub neutral.

Przykłady:
Tekst: "Świetny produkt, polecam wszystkim!"
Odpowiedź: {"sentiment": "positive"}

Tekst: "Okropna obsługa, nigdy więcej nie wrócę."
Odpowiedź: {"sentiment": "negative"}

Tekst: "Produkt jest w porządku, nic specjalnego."
Odpowiedź: {"sentiment": "neutral"}

{format_instructions}

Tekst do analizy:
{text}"""

PROMPTS = {
    "fewshot": PROMPT_FEW_SHOT
}

```

```

def setup_results_dirs() -> None:
    for subdir in ("predictions", "metrics", "logs", "reports"):
        (RESULTS_DIR / subdir).mkdir(parents=True, exist_ok=True)

def setup_logging() -> Path:
    log_path = RESULTS_DIR / "logs" / f"ewaluacja_{TIMESTAMP}.log"
    logging.basicConfig(
        filename=str(log_path),
        level=logging.INFO,
        format="%(asctime)s [%(levelname)s] %(message)s",
        encoding="utf-8",
    )
    return log_path

def safe_name(model_name: str) -> str:
    return model_name.replace("/", "_")

def make_config_id(prompt_kind: str, decoding: dict) -> str:
    mnt = decoding["max_new_tokens"]
    if decoding["do_sample"]:
        temp = decoding["temperature"]
        return f"{prompt_kind}_t{temp}_mnt{mnt}"
    return f"{prompt_kind}_greedy_mnt{mnt}"

def _to_polemo_label(label_name: str) -> str | None:
    """Mapuje nazwę sentymentu na klasę PolEmo2."""
    label = label_name.lower().strip()

    if label in ("very negative", "negative"):
        return "__label__meta_minus_m"
    if label in ("very positive", "positive"):
        return "__label__meta_plus_m"
    if label == "neutral":
        return "__label__meta_zero"
    if "negatywn" in label or "minus" in label:

```

```

        return "__label__meta_minus_m"
    if "pozytywn" in label or "plus" in label:
        return "__label__meta_plus_m"

    return None

def _keyword_fallback(raw_text: str) -> str:
    lower = raw_text.lower()
    if "positive" in lower or "pozytywn" in lower:
        return "positive"
    if "negative" in lower or "negatywn" in lower:
        return "negative"
    if "neutral" in lower or "neutraln" in lower:
        return "neutral"
    return "neutral"

def parse_model_output(raw_text: str) -> tuple[str, bool]:
    """Parsuje wyjście modelu przez JsonOutputParser; fallback na słowa kluczowe."""
    try:
        parsed = OUTPUT_PARSER.parse(raw_text)
        sentiment = str(parsed.get("sentiment", "")).strip()
        if sentiment:
            return sentiment, True
    except Exception:
        pass

    sentiment = _keyword_fallback(raw_text)
    logging.warning("Parse failed, keyword fallback: %s -> %s", raw_text[:80], sentiment)
    return sentiment, False

def map_sentiment(sentiment: str) -> str:
    mapped = _to_polemo_label(sentiment)
    if mapped:
        return mapped

    original = str(sentiment).strip()
    if original not in _UNKNOWN_SENTIMENTS:
        _UNKNOWN_SENTIMENTS.add(original)
        logging.warning("Nieznany sentyment: %s -> przypisano neutral", original)
    return "__label__meta_zero"

```

```

def build_pipeline(model_name: str):
    """Ładuje pipeline text-generation. Dla większych modeli użyj 4-bit (poniżej)."""
    device = 0 if torch.cuda.is_available() else -1
    dtype = torch.float16 if torch.cuda.is_available() else torch.float32

    pipe = pipeline(
        "text-generation",
        model=model_name,
        torch_dtype=dtype,
        device=device,
        trust_remote_code=False,
    )
    if pipe.tokenizer.pad_token is None:
        pipe.tokenizer.pad_token = pipe.tokenizer.eos_token
    return pipe

def build_chat_prompt(tokenizer, prompt_template: str, text: str) -> str:
    user_content = prompt_template.format(
        format_instructions=FORMAT_INSTRUCTIONS,
        text=text,
    )
    messages = [{"role": "user", "content": user_content}]
    return tokenizer.apply_chat_template(
        messages,
        tokenize=False,
        add_generation_prompt=True,
    )

def _generation_kwargs(decoding: dict) -> dict:
    kwargs = {
        "max_new_tokens": decoding["max_new_tokens"],
        "do_sample": decoding["do_sample"],
        "return_full_text": False,
    }
    if decoding["do_sample"] and decoding["temperature"] is not None:
        kwargs["temperature"] = decoding["temperature"]

```

```

return kwargs

def generate_and_parse(
    pipe,
    texts: list[str],
    prompt_template: str,
    decoding: dict,
    batch_size: int = GENERATION_BATCH_SIZE,
) -> list[dict]:
    """Generacja batchowa + parsowanie JsonOutputParser."""
    tokenizer = pipe.tokenizer
    gen_kwargs = _generation_kwargs(decoding)
    results = []

    for start in range(0, len(texts), batch_size):
        batch_texts = texts[start : start + batch_size]
        prompts = [
            build_chat_prompt(tokenizer, prompt_template, text) for text in batch_texts
        ]

        outputs = pipe(prompts, batch_size=len(prompts), **gen_kwargs)

        for output in outputs:
            if isinstance(output, list):
                raw = output[0].get("generated_text", "")
            else:
                raw = output.get("generated_text", "")

            sentiment, parse_ok = parse_model_output(raw)
            results.append(
                {
                    "raw_output": raw,
                    "predicted_sentiment": sentiment,
                    "mapped_prediction": map_sentiment(sentiment),
                    "parse_ok": parse_ok,
                }
            )

    return results

def sanity_check(
    pipe,
    dataset,
    prompt_template: str,
    model_name: str,
    n: int = SANITY_CHECK_N,
) -> None:
    """Mała próbka z wyświetleniem wyników przed pełnym przebiegiem."""
    decoding = {"do_sample": False, "temperature": None, "max_new_tokens": 64}
    sample = dataset.select(range(min(n, len(dataset))))
    texts = sample["sentence"]
    targets = sample["target"]

    print(f"\n{'=' * 60}")
    print(f"SANITY CHECK – model: {model_name}, próbka: {len(texts)} rekordów")
    print(f"\n{'=' * 60}")

    results = generate_and_parse(pipe, texts, prompt_template, decoding, batch_size=len(texts))

    for i, (text, target, res) in enumerate(zip(texts, targets, results), 1):
        preview = text[:120] + ("..." if len(text) > 120 else "")
        print(f"\n--- Rekord {i} ---")
        print(f"Wejście:      {preview}")
        print(f"Prawda:         {target}")
        print(f"Surowe out:     {res['raw_output']!r}")
        print(f"Sentymment:     {res['predicted_sentiment']!r}")
        print(f"Zmapowane:      {res['mapped_prediction']!r}")
        print(f"Parse OK:       {res['parse_ok']!r}")

    parse_rate = sum(r["parse_ok"] for r in results) / len(results)
    print(f"\nParse success rate (próbka): {parse_rate:.2%}")
    print(f"\n{'=' * 60}")
    logging.info("Sanity check zakończony: model=%s, parse_rate=%.2f", model_name, parse_rate)

def evaluate_config(
    pipe,
    dataset,
    model_name: str,
    prompt_kind: str,

```

```

prompt_template: str,
decoding: dict,
config_id: str,
) -> dict:
    texts = dataset["sentence"]
    true_labels = dataset["target"]

    logging.info(
        "Start: model=%s, config_id=%s, do_sample=%s, max_new_tokens=%d",
        model_name,
        config_id,
        decoding["do_sample"],
        decoding["max_new_tokens"],
    )

    parsed_results = generate_and_parse(pipe, texts, prompt_template, decoding)
    mapped_predictions = [r["mapped_prediction"] for r in parsed_results]
    parse_ok_flags = [r["parse_ok"] for r in parsed_results]
    n_parse_failed = sum(1 for ok in parse_ok_flags if not ok)
    parse_success_rate = sum(parse_ok_flags) / len(parse_ok_flags)

    accuracy = accuracy_score(true_labels, mapped_predictions)
    f1 = f1_score(true_labels, mapped_predictions, average="macro")
    recall = recall_score(true_labels, mapped_predictions, average="macro")
    precision = precision_score(true_labels, mapped_predictions, average="macro")

    run_timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    safe_model = safe_name(model_name)

    evaluation_summary = {
        "model": model_name,
        "prompt_kind": prompt_kind,
        "config_id": config_id,
        "do_sample": decoding["do_sample"],
        "temperature": decoding["temperature"],
        "max_new_tokens": decoding["max_new_tokens"],
        "timestamp": run_timestamp,
        "ogolne_metryki": {
            "accuracy": accuracy,
            "f1_macro": f1,
            "recall_macro": recall,
            "precision_macro": precision,
            "parse_success_rate": parse_success_rate,
            "n_parse_failed": n_parse_failed,
        },
        "detailed_report": classification_report(
            true_labels, mapped_predictions, output_dict=True
        ),
    }

    report_text = classification_report(true_labels, mapped_predictions)
    report_path = RESULTS_DIR / "reports" / f"{safe_model}_{config_id}_{run_timestamp}.txt"
    with open(report_path, "w", encoding="utf-8") as f:
        f.write(f"Model: {model_name}\n")
        f.write(f"Prompt: {prompt_kind}\n")
        f.write(
            f"Parametry: config_id={config_id}, do_sample={decoding['do_sample']}, "
            f"temperature={decoding['temperature']}, max_new_tokens={decoding['max_new_tokens']}\n"
        )
        f.write(f"Accuracy: {accuracy:.4f}\n")
        f.write(f"F1 macro: {f1:.4f}\n")
        f.write(f"Parse success rate: {parse_success_rate:.4f}\n\n")
        f.write("Szczegółowy raport (Quality per class):\n")
        f.write(report_text)

    json_path = RESULTS_DIR / "metrics" / f"{safe_model}_{config_id}_{run_timestamp}.json"
    with open(json_path, "w", encoding="utf-8") as f:
        json.dump(evaluation_summary, f, indent=4, ensure_ascii=False)

    df = dataset.to_pandas()
    df["raw_output"] = [r["raw_output"] for r in parsed_results]
    df["predicted_sentiment"] = [r["predicted_sentiment"] for r in parsed_results]
    df["mapped_prediction"] = mapped_predictions
    df["parse_ok"] = parse_ok_flags
    csv_path = RESULTS_DIR / "predictions" / f"wyniki_{safe_model}_{config_id}.csv"
    df.to_csv(csv_path, index=False, encoding="utf-8")

    logging.info(
        "Zakonczono: model=%s, config_id=%s, accuracy=%.4f, f1=%.4f, parse_rate=%.4f",
        model_name,
        config_id,
        accuracy,

```

```

        f1,
        parse_success_rate,
    )

    return {
        "model": model_name,
        "prompt_kind": prompt_kind,
        "config_id": config_id,
        "do_sample": decoding["do_sample"],
        "temperature": decoding["temperature"],
        "max_new_tokens": decoding["max_new_tokens"],
        "accuracy": accuracy,
        "f1_macro": f1,
        "recall_macro": recall,
        "precision_macro": precision,
        "parse_success_rate": parse_success_rate,
        "n_parse_failed": n_parse_failed,
        "timestamp": run_timestamp,
    }

def save_summary_csv(summary_rows: list[dict]) -> Path:
    summary_path = RESULTS_DIR / f"porownanie_llm_{TIMESTAMP}.csv"
    pd.DataFrame(summary_rows).to_csv(summary_path, index=False, encoding="utf-8")
    logging.info("Zaktualizowano zbiorcze porownanie: %s", summary_path)
    return summary_path

def download_partial(run_num: int) -> str:
    zip_base = str(RESULTS_DIR.parent / f"wyniki_llm_{TIMESTAMP}_run{run_num:02d}")
    zip_path = shutil.make_archive(zip_base, "zip", str(RESULTS_DIR))
    files.download(zip_path)
    print(f"Pobrano przyrostowy ZIP: {zip_path}")
    logging.info("Pobrano przyrostowy ZIP: %s", zip_path)
    return zip_path

def zip_and_download() -> str:
    zip_base = f"wyniki_llm_{TIMESTAMP}"
    zip_path = shutil.make_archive(zip_base, "zip", str(RESULTS_DIR))
    files.download(zip_path)
    print(f"Zapisano i pobrano finalny ZIP: {zip_path}")
    logging.info("Pobrano finalny ZIP: %s", zip_path)
    return zip_path

def unload_pipeline(pipe) -> None:
    del pipe
    gc.collect()
    if torch.cuda.is_available():
        torch.cuda.empty_cache()

def run_experiments(
    dataset,
    models: list[str],
    prompts: dict[str, str],
    grid: list[dict],
) -> pd.DataFrame:
    summary_rows = []
    total = len(models) * len(prompts) * len(grid)
    run_num = 0

    for model_name in models:
        logging.info("Ladowanie modelu: %s", model_name)
        pipe = build_pipeline(model_name)

        if RUN_SANITY_CHECK:
            sanity_check(
                pipe,
                dataset,
                prompts["fewshot"],
                model_name,
                n=SANITY_CHECK_N,
            )

        for prompt_kind, prompt_template in prompts.items():
            for decoding in grid:
                run_num += 1
                config_id = make_config_id(prompt_kind, decoding)
                logging.info("Eksperyment %d/%d: %s / %s", run_num, total, model_name, config_id)

```

```
        result = evaluate_config(
            pipe,
            dataset,
            model_name,
            prompt_kind,
            prompt_template,
            decoding,
            config_id,
        )
        summary_rows.append(result)
        save_summary_csv(summary_rows)

        if run_num % DOWNLOAD_EVERY_N == 0:
            download_partial(run_num)

    unload_pipeline(pipe)

    return pd.DataFrame(summary_rows)
```

```
setup_results_dirs()
setup_logging()
run_experiments(filtered_dataset, MODELS, PROMPTS, DECODING_GRID)
zip_and_download()
```